# Experiment 7

## Random Forest

**Aim:**

To implement the Random Forest algorithm for classification tasks and evaluate its performance using appropriate classification metrics.

**Theory:**

A Random Forest is a supervised ensemble learning algorithm used for both classification and regression tasks. It builds multiple decision trees during training and combines their outputs to produce a more accurate and stable prediction. Instead of relying on a single tree, Random Forest aggregates the results of many trees to reduce overfitting and improve generalization.

The main components of a Random Forest are:

1. Decision Trees: Individual models trained on different subsets of data.
2. Bootstrap Samples: Random subsets of the original dataset selected with re-placement.
3. Feature Randomness: A random subset of features is considered at each split to ensure diversity among trees.

Each tree in the forest makes a prediction, and the final output is determined by majority voting (for classification) or averaging (for regression).

A Random Forest works by constructing multiple decision trees using different bootstrap samples of the training data. For each tree, at every node, only a random subset of features is considered when selecting the best split. This randomness ensures that the trees are diverse and not highly correlated. The splitting criteria commonly used include Gini Index and Entropy. The Gini Index measures the impurity of a node, representing the probability of incorrectly classifying a randomly chosen sample if it were labeled according to the class distribution in the node. The Gini Index is calculated as:

$$\text{Gini}(S) = 1 - \sum_{i=1}^{n} p_i^2$$

Lower Gini values indicate purer nodes, with a value of zero when all samples belong to one class. However, it is slightly biased toward dominant classes.

Entropy, on the other hand, measures the amount of uncertainty or disorder in a node and is used to calculate Information Gain and is computed as:

$$\text{Entropy}(S) = -\Sigma\, p_i \log_2(p_i)$$

Entropy is zero for a perfectly pure node, with higher values indicating greater disorder. It is sensitive to small changes in class distribution and often produces balanced and informative splits. Once all trees are trained, predictions are made by combining the outputs of individual trees. In classification, the class predicted by the majority of trees is selected, while in regression, the average of all tree predictions is taken as the final output. By combining multiple trees, Random Forest reduces variance, minimizes overfitting, and provides higher accuracy compared to a single decision tree.

**Merits of Random Forest**

1. **High Accuracy & Robustness** – By combining multiple decision trees, Random Forest reduces overfitting and usually gives better accuracy than a single decision tree.

2. **Handles Large & Complex Data** – It works well with large datasets and can handle both numerical and categorical features effectively.

3. **Feature Importance Estimation** – Random Forest can measure the importance of each feature, helping in feature selection and understanding the model.

**Demerits of Random Forest**

1. **Computationally Expensive** – Training many trees requires more time and memory compared to simpler models.

2. **Less Interpretability** – The ensemble naturemakes it difficult to interpret compared to a single decision tree.

3. **Not Ideal for Real-Time Prediction** – Due to multiple trees, predictioncan be slower, making it less suitable for time-critical applications.

**Algorithm**

**Step 1:** Set parameters:

- n = number of trees to build
- m = number of features to consider at each split (usually √total_features)

**Step 2:** For each tree i = 1 to n:
**Step 2a: Bootstrap Sampling**

- Randomly select N samples with replacement from training data
- This creates a different dataset for each tree
- About 37% of original data is left out (Out-of-Bag samples)

**Step 2b: Build Decision Tree with Random Feature Selection**

- At each node:
    - Randomly select m features from all features
    - Find the best split using ONLY these m features
    - Split the node
- Grow tree fully (no pruning)

**Step 3:** Store all n trees
**Step 4:** For prediction (Classification):

- Pass new sample through all trees
- Each tree gives one vote
- Final prediction = class with majority votes

**Step 5:** For prediction (Regression):

- Pass new sample through all trees
- Each tree gives one numeric prediction
- Final prediction is calculated as the average of all tree predictions

**Step 6:** Calculate Feature Importance:

- For each feature, sum up how much it reduced impurity across all trees
- Normalize to get importance scores

# Random Forest - Practical Implementation

*Virtual Labs - Dayalbagh Educational Institute*

## Option A: Binary Classification (Bank Marketing Dataset)

### Step 1: Importing Libraries

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

print("Libraries imported successfully")
```

**Output:**

```
Libraries imported successfully
```

### Step 2: Loading Dataset

**# Import necessary libraries**

```
df = pd.read_csv("bank_marketing.csv", sep=';')

print("Dataset loaded successfully")
```

**Output:**

```
Dataset loaded successfully
```

### Step 3: Data Analysis

**# Check dataset dimensions**

```
print("Dataset shape:", df.shape)
```

**Output:**

```
Dataset shape: (41188, 21)
```

**# Preview first 5 records of the dataset**

```
print("First 5 rows:")
print(df.head())
```

**Output:**

```
First 5 rows:
```

**Output:**

| | age | job | marital | education | default | housing | loan | contact | ... |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 56 | housemaid | married | basic.4y | no | no | no | telephone | ... |
| 1 | 57 | services | married | high.school | unknown | no | no | telephone | ... |
| 2 | 37 | services | married | high.school | no | yes | no | telephone | ... |
| 3 | 40 | admin. | married | basic.6y | no | no | no | telephone | ... |
| 4 | 56 | services | married | high.school | no | no | yes | telephone | ... |

**Output:**

```
[5 rows x 21 columns]
```

## # Check dataset information and types

```
print("Dataset information:")
df.info()
```

**Output:**

```
Dataset information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 41188 entries, 0 to 41187
Data columns (total 21 columns):
#   Column          Non-Null Count  Dtype
---  ------          --------------  -----
0   age             41188 non-null  int64
1   job             41188 non-null  object
2   marital         41188 non-null  object
3   education       41188 non-null  object
4   default         41188 non-null  object
5   housing         41188 non-null  object
6   loan            41188 non-null  object
7   contact         41188 non-null  object
8   month           41188 non-null  object
9   day_of_week     41188 non-null  object
10  duration        41188 non-null  int64
11  campaign        41188 non-null  int64
12  pdays           41188 non-null  int64
13  previous        41188 non-null  int64
14  poutcome        41188 non-null  object
15  emp.var.rate    41188 non-null  float64
16  cons.price.idx  41188 non-null  float64
17  cons.conf.idx   41188 non-null  float64
18  euribor3m       41188 non-null  float64
19  nr.employed     41188 non-null  float64
20  y               41188 non-null  object
dtypes: float64(5), int64(5), object(11)
memory usage: 6.6+ MB
```

## Step 4: Data Preprocessing

### # Perform Label Encoding on categorical variables

```
df_encoded = df.copy()
le = LabelEncoder()

for col in df_encoded.columns:
    if df_encoded[col].dtype == 'object':
        df_encoded[col] = le.fit_transform(df_encoded[col])
        print(f"Encoded column: {col}")

print("\\nCategorical encoding completed")
```

**Output:**

```
Encoded column: job
Encoded column: marital
Encoded column: education
Encoded column: default
Encoded column: housing
Encoded column: loan
Encoded column: contact
Encoded column: month
Encoded column: day_of_week
Encoded column: poutcome
Encoded column: y
Categorical encoding completed
```

### # Separate independent and dependent variables

```
X = df_encoded.drop('y', axis=1)
y = df_encoded['y']

print("Features and target separated")
```

**Output:**

```
Features and target separated
```

### # Split data into Training and Testing sets

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print(f"Training set: {len(X_train)} samples")
print(f"Testing set: {len(X_test)} samples")
```

**Output:**

```
Training set: 32950 samples
Testing set: 8238 samples
```

# Option A-i: Binary - Gini Impurity

## Step 5: Model Training

### # Initialize Random Forest classifier with Gini criterion

```
rf_model = RandomForestClassifier(
    n_estimators=200,
    max_depth=15,
    min_samples_split=5,
    min_samples_leaf=2,
    random_state=42,
    n_jobs=-1
)

print("Random Forest model initialized with Gini criterion")
```

**Output:**

```
Random Forest model initialized with Gini criterion
```

### # Fit the model on training data

```
rf_model.fit(X_train, y_train)

print("Model training completed")
print(rf_model)
```

**Output:**

```
Model training completed
RandomForestClassifier(max_depth=15, min_samples_leaf=2, min_samples_split=5, n_estimators=200,
n_jobs=-1, random_state=42)
```

## Step 6: Model Evaluation

### # Generate predictions on test data

```
y_pred = rf_model.predict(X_test)

print(f"Predictions generated for {len(y_pred)} test samples")
```

**Output:**

```
Predictions generated for 8238 test samples
```

### # Calculate overall accuracy of the model

```
accuracy = accuracy_score(y_test, y_pred)

print(f"Accuracy of Random Forest model: {accuracy}")
```

Output:
```
Accuracy of Random Forest model: 0.9225540179655256
```

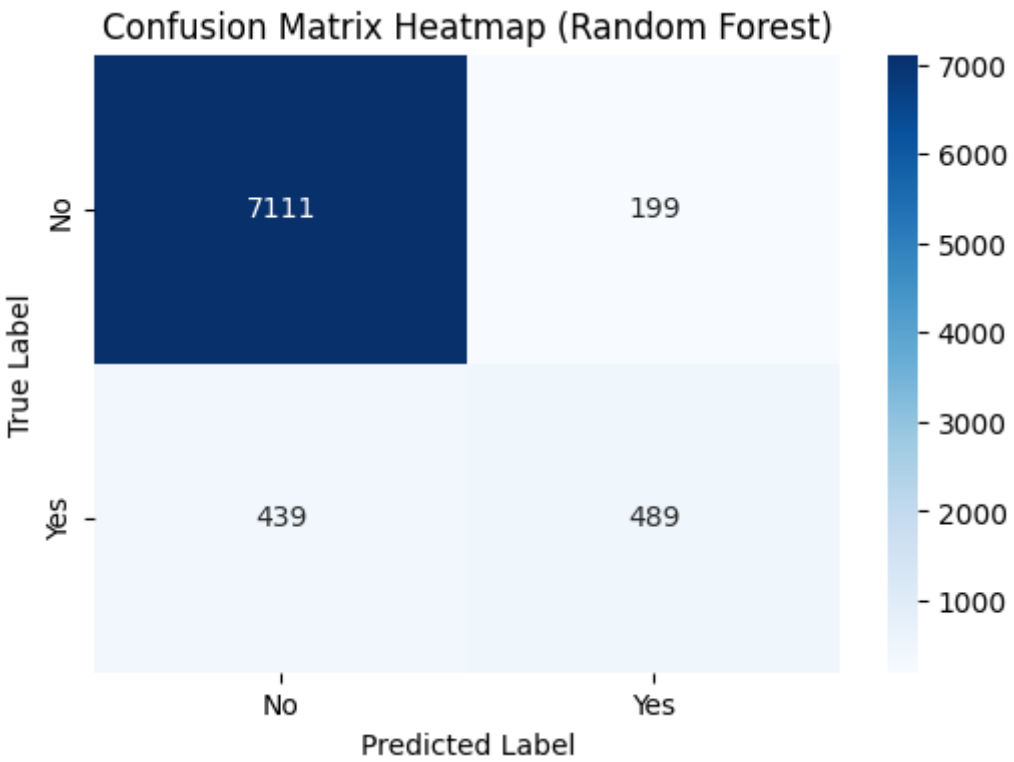
## # Display detailed classification report

```
print(classification_report(y_test, y_pred))
```

Output:
```
Classification Report:
precision    recall  f1-score   support
0       0.94      0.97      0.96      7310
1       0.71      0.53      0.61       928
accuracy                           0.92      8238
macro avg       0.83      0.75      0.78      8238
weighted avg       0.92      0.92      0.92      8238
```

```
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6, 4))
sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=['No', 'Yes'],
    yticklabels=['No', 'Yes']
)
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.title("Confusion Matrix Heatmap (Random Forest)")
plt.show()
```



## Step 7: Model Simulation

**# Test prediction on sample data**

```
sample_data = [[56, 1, 1, 0, 0, 0, 0, 1, 0, 0, 261, 1, 999, 0, 0, 1.1, 93.994, -36.4,
    4.857, 5191.0]]

prediction = rf_model.predict(sample_data)

if prediction[0] == 0:
    print("Prediction: No (Customer will NOT subscribe)")
else:
    print("Prediction: Yes (Customer will subscribe)")
```

**Output:**

```
Prediction: No (Customer will NOT subscribe)
```

---

# Option A-ii: Binary - Entropy

## Step 5: Model Training

**# Initialize Random Forest classifier with Entropy criterion**

```
rf_entropy = RandomForestClassifier(
    criterion='entropy',
    random_state=42,
    n_estimators=200,
    max_depth=15,
    min_samples_split=5,
    min_samples_leaf=2,
    n_jobs=-1
)

print("Random Forest model initialized with Entropy criterion")
```

**Output:**

```
Random Forest model initialized with Entropy criterion
```

**# Fit the model on training data**

```
rf_entropy.fit(X_train, y_train)

print("Model training completed")
```

**Output:**

```
Model training completed
```

## Step 6: Model Evaluation

### # Generate predictions on test data

```
y_pred = rf_entropy.predict(X_test)

print(f"Predictions generated for {len(y_pred)} test samples")
```

**Output:**

```
Predictions generated for 8238 test samples
```

### # Calculate overall model accuracy

```
accuracy = accuracy_score(y_test, y_pred)

print(f"Overall Accuracy: {accuracy:.4f}")
```

**Output:**

```
Overall Accuracy: 0.9226
```

### # Display detailed classification report

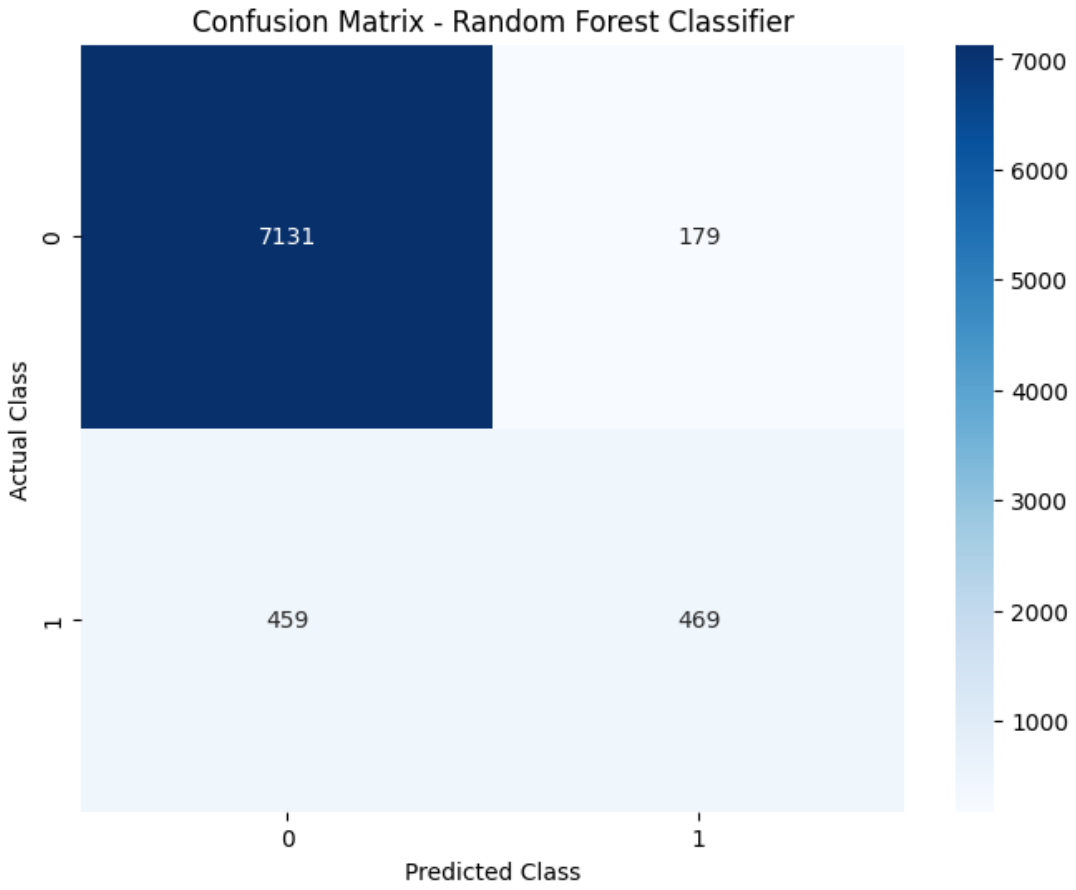
```
print(classification_report(y_test, y_pred))
```

**Output:**

```
Classification Report:
precision    recall  f1-score   support
0       0.94      0.98      0.96      7310
1       0.72      0.51      0.60       928
accuracy                           0.92      8238
macro avg       0.83      0.74      0.78      8238
weighted avg       0.92      0.92      0.92      8238
```

```
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(8, 6))
sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=['No', 'Yes'],
    yticklabels=['No', 'Yes']
)
plt.xlabel("Predicted Class")
plt.ylabel("Actual Class")
plt.title("Confusion Matrix - Random Forest Classifier")
plt.show()
```




## Step 7: Model Simulation

**# Test prediction on sample data**

```
sample_data = [[56, 1, 1, 0, 0, 0, 0, 1, 0, 0, 261, 1, 999, 0, 0, 1.1, 93.994, -36.4,
    4.857, 5191.0]]

prediction = rf_entropy.predict(sample_data)

if prediction[0] == 0:
    print("Prediction: No (Customer will NOT subscribe)")
else:
    print("Prediction: Yes (Customer will subscribe)")
```

**Output:**

```
Prediction: No (Customer will NOT subscribe)
```

---

## Option B: Multiclass Classification (Seeds Dataset)

## Step 1: Importing Libraries

### # Import core data handling libraries

```
import pandas as pd
import numpy as np

print("Pandas and Numpy imported")
```

**Output:**
```
Pandas and Numpy imported
```

### # Import machine learning models and utilities

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

print("Scikit-learn modules imported")
```

**Output:**
```
Scikit-learn modules imported
```

### # Import visualization tools

```
import matplotlib.pyplot as plt
import seaborn as sns

print("Visualization libraries imported")
```

**Output:**
```
Visualization libraries imported
```

## Step 2: Loading Dataset

### # Define dataset column names

```
column_names = [
    "Area", "Perimeter", "Compactness",
    "Length_of_kernel", "Width_of_kernel",
    "Asymmetry_coefficient", "Length_of_kernel_groove", "Class"
]

print("Column names defined:", column_names)
```

**Output:**
```
Column names defined: ["Area", "Perimeter", "Compactness", "Length_of_kernel",
"Width_of_kernel", "Asymmetry_coefficient", "Length_of_kernel_groove", "Class"]
```

### # Load Seeds dataset from local file

```
df = pd.read_csv("seeds_dataset.txt", sep=r"\\s+", header=None, names=column_names)

print("Dataset loaded from seeds_dataset.txt")
```

**Output:**

```
Dataset loaded from seeds_dataset.txt
```

## Step 3: Data Analysis

### # Check dataset dimensions

```
print("Dataset shape:", df.shape)
```

**Output:**

```
Dataset shape: (210, 8)
```

### # Preview first 5 records of the dataset

```
print("First 5 rows:")
print(df.head())
```

**Output:**

```
First 5 rows:
```

**Output:**

| | Area | Perimeter | Compactness | Length_of_kernel | Width_of_kernel | Asymmetry_coefficient | Length_of_kernel_groove | Class |
|---|---|---|---|---|---|---|---|---|
| 0 | 15.26 | 14.84 | 0.8710 | 5.763 | 3.312 | 2.221 | 5.220 | 1 |
| 1 | 14.88 | 14.57 | 0.8811 | 5.554 | 3.333 | 1.018 | 4.956 | 1 |
| 2 | 14.29 | 14.09 | 0.9050 | 5.291 | 3.337 | 2.699 | 4.825 | 1 |
| 3 | 13.84 | 13.94 | 0.8955 | 5.324 | 3.379 | 2.259 | 4.805 | 1 |
| 4 | 16.14 | 14.99 | 0.9034 | 5.658 | 3.562 | 1.355 | 5.175 | 1 |

### # Check the distribution of target classes

```
print("Class distribution:")
print(df["Class"].value_counts())
```

**Output:**

```
Class distribution:
1    70
2    70
3    70
Name: Class, dtype: int64
```

### # Generate descriptive statistical summary

```
print("Statistical summary:")
print(df.describe())
```

**Output:**

```
Statistical summary:
```

Output:

| | Area | Perimeter | Compactness | Length_of_kernel | Width_of_kernel | Asymmetry_coefficient | Length_of_kernel_groove | Class |
|---|---|---|---|---|---|---|---|---|
| count | 210.000000 | 210.000000 | 210.000000 | 210.000000 | 210.000000 | 210.000000 | 210.000000 | 210.000000 |
| mean | 14.847524 | 14.559286 | 0.870999 | 5.628533 | 3.258605 | 3.700201 | 5.408071 | 2.000000 |
| std | 2.909699 | 1.305959 | 0.023629 | 0.443063 | 0.377714 | 1.503557 | 0.491480 | 0.818448 |
| min | 10.590000 | 12.410000 | 0.808100 | 4.899000 | 2.630000 | 0.765100 | 4.519000 | 1.000000 |
| 25% | 12.270000 | 13.450000 | 0.856900 | 5.262250 | 2.944000 | 2.561500 | 5.045000 | 1.000000 |
| 50% | 14.355000 | 14.320000 | 0.873450 | 5.523500 | 3.237000 | 3.599000 | 5.223000 | 2.000000 |
| 75% | 17.305000 | 15.715000 | 0.887775 | 5.979750 | 3.561750 | 4.768750 | 5.877000 | 3.000000 |
| max | 21.180000 | 17.250000 | 0.918300 | 6.675000 | 4.033000 | 8.456000 | 6.550000 | 3.000000 |

## Step 4: Data Preprocessing

**# Separate features and target variable**

```
X = df.drop("Class", axis=1)
y = df["Class"]

print("Features and target separated")
```

Output:

```
Features and target separated
```

**# Split data into training (75%) and testing (25%) sets**

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

print(f"Training set: {len(X_train)} samples")
print(f"Testing set: {len(X_test)} samples")
```

Output:

```
Training set: 157 samples
Testing set: 53 samples
```

# Option B-i: Multiclass - Gini Impurity

## Step 5: Model Training

**# Initialize Random Forest with GINI criterion**

```
rf_gini = RandomForestClassifier(n_estimators=100, criterion="gini", random_state=42)

print("Random Forest model (Gini) initialized")
```

Output:
```
Random Forest model (Gini) initialized
```

```
# Train Random Forest model using GINI
```

```
rf_gini.fit(X_train, y_train)

print("Model training completed")
print(rf_gini)
```

Output:
```
Model training completed
RandomForestClassifier(random_state=42)
```

## Step 6: Model Evaluation

```
# Calculate and display GINI model accuracy
```

```
accuracy = accuracy_score(y_test, rf_gini.predict(X_test))

print(f"Model accuracy: {accuracy}")
```

Output:
```
Model accuracy: 0.9056603773584906
```

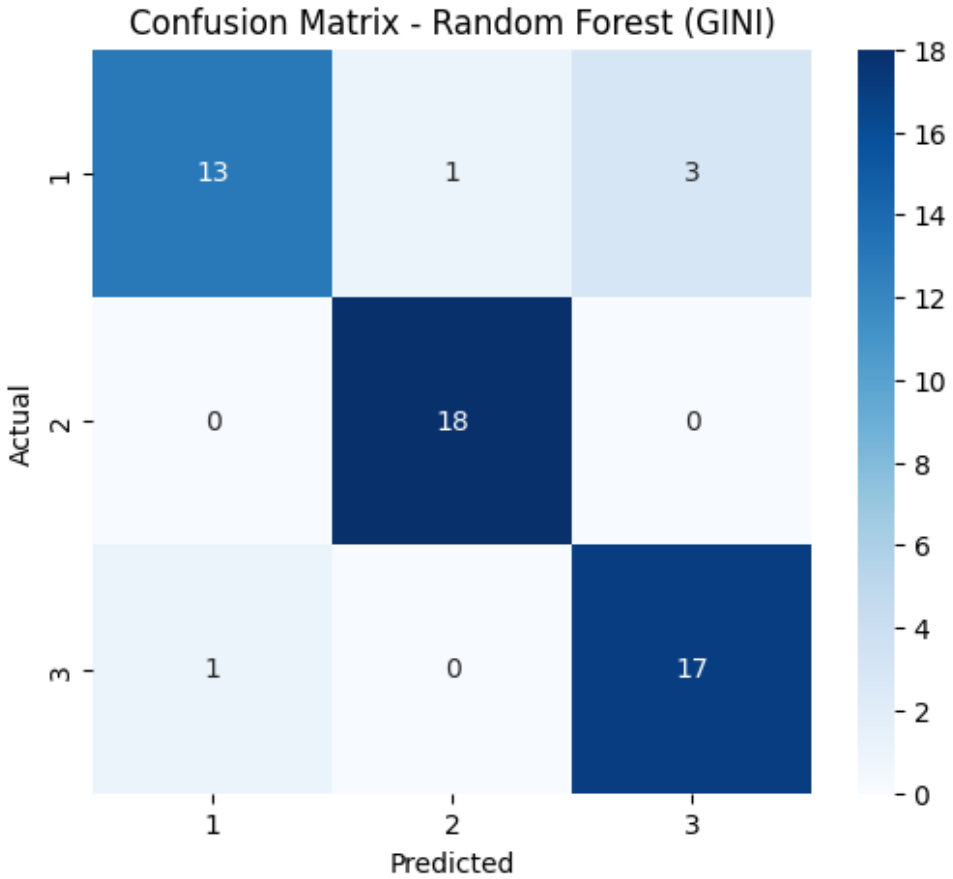
```
# Display detailed classification report for GINI model
```

```
print(classification_report(y_test, rf_gini.predict(X_test)))
```

Output:
```
precision    recall  f1-score   support
1       0.93      0.76      0.84        17
2       0.95      1.00      0.97        18
3       0.85      0.94      0.89        18
accuracy                           0.91        53
macro avg       0.91      0.90      0.90        53
weighted avg       0.91      0.91      0.90        53
```

```
# Plot confusion matrix for GINI model predictions
```

```
cm_gini = confusion_matrix(y_test, rf_gini.predict(X_test))
sns.heatmap(cm_gini, annot=True, fmt="d", cmap="Blues")
plt.title("Confusion Matrix - GINI")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```




## Step 7: Model Simulation

**# Select sample test indices representing different classes**

```
sample_idx = [0, 20, 40]

print("Sample indices selected:", sample_idx)
```

**Output:**

```
Sample indices selected: [0, 20, 40]
```

**# Perform manual prediction and compare with true labels**

```
samples = X_test.iloc[sample_idx]
true_labels = y_test.iloc[sample_idx]
preds = rf_gini.predict(samples)

for i in range(len(sample_idx)):
    print(f"Sample {i+1} Prediction: {preds[i]} (True: {true_labels.iloc[i]})")
```

**Output:**

```
Sample 1 Prediction: 1 (True: 1)
Sample 2 Prediction: 2 (True: 2)
Sample 3 Prediction: 3 (True: 3)
```

# Option B-ii: Multiclass - Entropy

## Step 5: Model Training

### # Initialize Random Forest with ENTROPY criterion

```
rf_entropy = RandomForestClassifier(n_estimators=100, criterion="entropy",
    random_state=42)

print("Random Forest model (Entropy) initialized")
```

**Output:**

```
Random Forest model (Entropy) initialized
```

### # Train Random Forest model using ENTROPY

```
rf_entropy.fit(X_train, y_train)

print("Model training completed")
print(rf_entropy)
```

**Output:**

```
Model training completed
RandomForestClassifier(criterion='entropy', random_state=42)
```

## Step 6: Model Evaluation

### # Calculate and display ENTROPY model accuracy

```
accuracy = accuracy_score(y_test, rf_entropy.predict(X_test))

print(f"Model accuracy: {accuracy}")
```

**Output:**

```
Model accuracy: 0.8867924528301887
```

### # Display detailed classification report for ENTROPY model

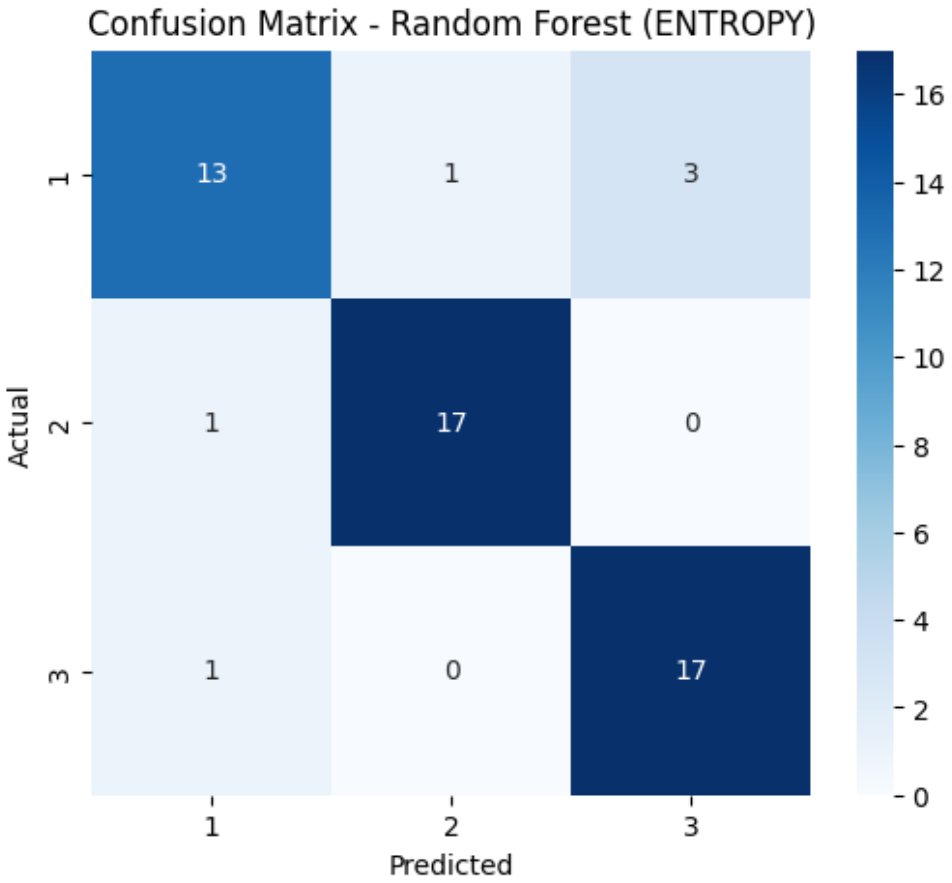
```
print(classification_report(y_test, rf_entropy.predict(X_test)))
```

**Output:**

```
Classification Report:
precision    recall  f1-score   support
1       0.88      0.82      0.85        17
2       0.95      1.00      0.97        18
3       0.84      0.83      0.83        18
accuracy                           0.89        53
macro avg       0.89      0.89      0.88        53
weighted avg       0.89      0.89      0.89        53
```

**# Plot confusion matrix for ENTROPY model predictions**

```
cm_entropy = confusion_matrix(y_test, rf_entropy.predict(X_test))
sns.heatmap(cm_entropy, annot=True, fmt="d", cmap="Blues")
plt.title("Confusion Matrix - ENTROPY")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```




## Step 7: Model Simulation

**# Select sample test indices representing different classes**

```
sample_idx = [0, 20, 40]

print("Sample indices selected:", sample_idx)
```

**Output:**

```
Sample indices selected: [0, 20, 40]
```

**# Perform manual prediction and compare with true labels**

```
samples = X_test.iloc[sample_idx]
true_labels = y_test.iloc[sample_idx]
preds = rf_entropy.predict(samples)

for i in range(len(sample_idx)):
    print(f"Sample {i+1} Prediction: {preds[i]} (True: {true_labels.iloc[i]})")
```

**Output:**

```
Sample 1 Prediction: 1 (True: 1)
Sample 2 Prediction: 2 (True: 2)
Sample 3 Prediction: 3 (True: 3)
```